

L'essentiel du diagnostic

La boîte à outils : cinq commandes qui répondent à cinq questions

Un conteneur qui ne marche pas, ça se diagnostique toujours de la même façon : on pose une question, on tape la commande qui y répond, on avance. Voilà la boîte à outils de base, dont l'essentiel a déjà servi en J1.

```
# Qu'est-ce qui tourne, et depuis quand ?
docker ps -a

# Qu'est-ce que l'application a écrit avant de mourir ?
docker logs mon-conteneur
docker compose logs -f api
docker compose logs --tail=100 --since 30m api

# Comment Docker voit-il ce conteneur (état, réseau, variables) ?
docker inspect mon-conteneur
docker inspect --format='{{.State.Status}}' mon-conteneur
docker inspect --format='{{.NetworkSettings.IPAddress}}' mon-conteneur

# Qu'est-ce que ça consomme, là, maintenant ?
docker stats --no-stream
docker top mon-conteneur

# Et si j'allais voir de l'intérieur ?
docker exec -it mon-conteneur sh
docker run -it --entrypoint sh mon-image # même quand le conteneur crashe au démarrage
```

Deux d'entre elles méritent un mot, parce qu'on ne les a pas croisées en J1.

docker stats ? Un affichage en temps réel de ce que consomment les conteneurs qui tournent : CPU, mémoire, réseau, disque. Avec **--no-stream**, on prend une photo à l'instant t au lieu d'un flux continu. C'est la commande qu'on dégage quand une machine rame sans qu'on sache lequel des huit conteneurs est en cause.

--entrypoint sh ? Une façon de démarrer une image en remplaçant la commande de lancement par un shell. Quand un conteneur crashe instantanément, on ne peut pas faire **docker exec** dessus (il n'existe déjà plus). Là, on entre dans l'image telle quelle et on regarde ce qu'il y a dedans : les fichiers sont-ils au bon endroit, le **node_modules** existe-t-il vraiment.

Les erreurs qui reviennent tout le temps

Ces six messages représentent l'écrasante majorité des blocages Docker en début de parcours. Le réflexe à avoir : lire le message *en entier*, le reconnaître, appliquer la contre-mesure.

Erreur	Cause	Solution
<code>port already allocated</code>	Port déjà utilisé sur l'hôte	<code>lsof -i :3000</code> , changer le port hôte

Erreur	Cause	Solution
<code>no space left on device</code>	Disque plein	<code>docker system df</code> , puis <code>docker system prune -a</code>
<code>network not found</code>	Network supprimé ou inexistant	<code>docker compose down && docker compose up</code>
<code>service unhealthy</code>	Healthcheck échoue	<code>docker logs</code> + <code>docker inspect</code> pour voir le log du healthcheck
<code>connection refused</code>	Service pas encore prêt	Ajouter <code>depends_on</code> avec <code>service_healthy</code>
<code>unable to find image</code>	Typo ou image inexistante	Vérifier le nom, <code>docker pull</code>

Instant Cheat Sheet! Ce tableau, c'est le premier réflexe de la journée. Une pipeline qui casse cet après-midi affichera exactement les mêmes messages, à un détail près : ils défileront dans les logs d'un runner GitHub au lieu de votre terminal. Le diagnostic, lui, ne change pas.

Quatre fichiers cassés, quatre diagnostics

On y va. Pour chacun, une entrée dans le Journal de bord : *le symptôme observé, la cause, la correction*.

Fichier 1 : l'erreur subtile

```
FROM node:18-alpine
WORKDIR /app
COPY package.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD npm start
```

On le build avec `docker build -t debug-1 .` et on regarde ce qui se passe. Deux pistes pour le Journal de bord : ce qui manque dans le `COPY` de la ligne 3 par rapport à ce qu'on a écrit en J1, et pourquoi la forme du `CMD` de la dernière ligne pose problème quand on veut arrêter le conteneur proprement.

Fichier 2 : l'ordre compte

```
FROM node:18-alpine
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 3000
CMD ["node", "server.js"]
```

Celui-ci fonctionne. Le problème n'est pas un plantage, c'est une lenteur. Le protocole : on build une première fois, on chronomètre, on modifie une ligne dans un fichier source, on rebuild, on chronomètre à nouveau. Ce qu'il faut noter dans le Journal de bord : les deux durées, et l'explication du deuxième chiffre à partir du cache de layers vu en J1.

Fichier 3 : l'image géante

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

On build, puis `docker images debug-3` pour lire la taille. Elle dépasse le gigaoctet. Dans le Journal de bord : au moins trois raisons distinctes à ce poids, et pour chacune, la technique de J1 qui la corrige.

Fichier 4 : le compose qui ne se parle pas

```
services:
  web:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      - database
    environment:
      DB_HOST: postgres
      DB_PORT: 5432

  database:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data
```

Celui-là a deux défauts, et ils ne se manifestent pas au même moment. Le premier empêche le fichier d'être lu du tout. Le second laisse la stack démarrer, mais l'application n'atteint jamais sa base de données. Le lancement se fait avec `docker compose -f docker-compose-broken.yml up`, et hop les deux diagnostics vont dans le Journal de bord.

Ptit tips de nouveauté : ce fichier a un troisième défaut, invisible à l'exécution. Beaucoup de `docker-compose.yml` qu'on trouve en ligne commencent par une ligne `version: '3.8'`. Elle ne sert plus à rien depuis des années, et Docker Compose affiche désormais un avertissement quand il la rencontre. Si on la voit dans un fichier, on la retire.